IMF2SQL – IMF to SQL via Text-to-SQL Translation

Sia Arabandi, Durga (Pranati) Duvvuri, Eileen Chen, Veda Gavinolla, John (Will) Neubauer, Aesha Shah, Erika Tay

alphabetically ordered by last name

Abstract

Accessing structured economic data through natural language interfaces has become increasingly important as large language models (LLMs) continue to improve. Institutions such as the International Monetary Fund (IMF) publish extensive statistical tables that span indicators, countries, and time periods, yet these datasets are often fragmented across spreadsheets and exhibit inconsistent formatting. This makes direct querying difficult, especially for non-technical users. In this project, we develop an end-to-end system that enables natural-language queries over IMF statistical releases. Our pipeline consolidates raw IMF tables into a unified SQLite database, applies lightweight schema normalization, and generates SQL queries using a prompting strategy tailored to economic data. We also create a set of 110 natural-language questions based on templated patterns that vary in structural complexity. These questions serve as a controlled way to evaluate how reliably an LLM can map economic questions to executable SQL. Finally, the system executes the generated SQL and produces a narrative explanation of the results. Together, these components form a practical framework for studying text-to-SQL performance on real-world macroeconomic data and highlight the challenges that arise when applying LLMs to noisy, heterogeneous institutional datasets.

1 Introduction

The ability to interact with structured data using natural language, rather than formal query syntax, has become a defining capability of modern large language models (LLMs) (Rajkumar et al., 2022). As organizations continue to generate high-dimensional and heterogeneous datasets, text-to-SQL systems play an increasingly important role in democratizing access to data and enabling non-technical users to retrieve meaningful information (Yu et al., 2018). This need is particularly acute in the field of macroeconomics, where institutions such as the International Monetary Fund (IMF) publish extensive statistical tables covering indicators, countries, and time periods. Despite their analytical richness, these datasets are difficult to query directly: they are distributed across numerous Excel or CSV files, formatted inconsistently, and follow schema conventions that can vary widely across releases (International Monetary Fund, 2024).

Although the text-to-SQL literature has made substantial progress, through benchmark development (Yu et al., 2018), cross-domain generation techniques (Zhang et al., 2023), and schema-aware prompting methods, real-world economic datasets pose additional challenges. First, macroeconomic data does not follow a standardized relational structure: each IMF table uses its own naming conventions, time formats, and indicator breakdowns. Second, natural-language questions about economics often require multi-step reasoning, involving temporal comparisons, multi-indicator relationships, or cross-country analysis. Finally, existing benchmarks such as Spider provide clean, curated academic schemas that differ substantially from noisy government or institutional data. As a result, it remains difficult to evaluate how well LLM-based SQL generators generalize to real-world macroeconomic contexts.

To address this gap, we develop an end-to-end pipeline that allows users to pose natural-language questions over IMF statistical datasets. The system processes raw IMF files, converts them into a unified SQLite database, generates SQL using a structured prompting strategy, executes the resulting queries, and produces natural-language explanations of the retrieved results. We also construct a set of 110 natural-language questions derived from templated patterns, which vary in difficulty based on SQL operations, join requirements, and nested query structures. These questions allow us to study LLM behavior across

different levels of complexity.

Our contributions are:

- We construct a unified relational database from raw IMF statistical tables and apply consistent schema normalization to support reliable SQL generation.
- We design an evaluation set of 110 natural-language questions derived from reusable templates, each categorized by SQL complexity.
- We develop a complete text-to-SQL pipeline, including schema extraction, SQL generation, execution, and answer verbalization, tailored to the characteristics of IMF data.

Together, these components form a practical framework for exploring text-to-SQL generation on large-scale, heterogeneous economic data and highlight the challenges that arise when applying LLMs beyond academic benchmarks.

2 Related Works

Several recent works in the text-to-SQL field and large language modeling shaped the design and pipeline of our system for querying the IMF database. The Spider (Yu et al., 2018) benchmark introduces the standard for cross domain tests to SQL evaluation, which also emphasizes the generalization across heterogeneous schemas rather than memorization within a single domain. The evaluation protocols which included structural awareness guided our decision to treat the IMF dataset as a multi-schema environment and motivated the use of schema serialization and table linking strategies in our prompt design.

Additionally, traditional seq2seq (encoder $\rightarrow$ decoder) models often fail on text-to-SQL because they try to output the whole query in one go. Due to this limitation, DCG-SQL (Lee et al., 2025) proposed a dependency-based compositional approach, generating SQL queries through structured immediate steps. This demonstrated that pipeline decomposition reduces logical and syntactic errors in complex queries. Our pipeline incorporates a similar stage to the generation process: the system identifies relevant IMF indicators, join operations, and constraints before producing the final SQL output, mirroring the compositional generation principles in DCG-SQL.

In further discussion, one study (Zhang et al. 2023) examined LLM behavior across datasets and schema complexities. The author showed that schema serialization, explicit column descriptions, and carefully chosen examples substantially improve performance, especially in specialized domains. These findings helped us make two design choices in our pipeline: (1) embedding IMF table metadata directly into prompts, and (2) standardizing naming conventions to reduce semantic ambiguity across economic indicator tables. Specifically in our implementation, we normalize the IMF table and column names through automated pre-processing, mirroring schema canonicalization techniques recommended in the evaluation study.

Finally, UNITE (Lan et al., 2023) introduced a unified framework that combines retrieval, schema linking, and stepwise query planning. A central insight of the work is that LLMs benefit from grounding SQL generation and domain-specific textual context. Our pipeline follows this principle partly. Although we do not yet Implement a full retrieval module, we ensure that the model receives clear, canonicalized domain information through structured schema prompts. By incorporating retrieved IMF documentation such as indicator definitions or methodological notes, we further align our system with UNITE's retrieval-augmented architecture.

Together, these works (and more) created the foundation for our system. Spider provided the evaluation model, DCG-SQL influenced compositional generation steps, the large scale benchmarking paper guided schema aware prompting, and UNITE informed the retrieval-augmented architecture. Their combined insights enabled a pipeline that aligns IMF domain knowledge with reliable SQL generation

3 Problem Definition

The goal of this project is to enable natural-language access to heterogeneous macroeconomic data released by the International Monetary Fund (IMF). Given a user question expressed in plain English, the system must generate an accurate SQL query that can be executed over a consolidated relational database constructed from raw IMF statistical files. The database consists of multiple tables, each corresponding to a separate IMF dataset, and contains indicators, country identifiers, and time-series measurements.

Formally, the task can be described as follows:

- **Input:** A natural-language question q written by the user, and a database schema S generated from IMF CSV files.
- **Output:** A single SQL query Q that is syntactically valid, uses only tables and columns present in S, and retrieves the information requested in the question. After execution, the system also provides a natural-language explanation of the results.

The task is complicated by several characteristics of IMF data: inconsistent naming conventions, non-standardized table structures, and questions that require multi-step reasoning. Our system addresses these challenges through schema normalization, explicit schema serialization, and a two-stage workflow for generating SQL and verbalizing results.

4 Proposed Method

Our approach consists of two major components: (1) constructing a unified SQL database from raw IMF spreadsheets, and (2) designing a query-processing pipeline that converts natural-language questions into executable SQL and interprets the results. The implementation follows the behavior of the project's codebase, including `creating_database.py`, `imf_nlqa.py`, and `run_pipeline.py`.

4.1 Database Structuring

The first stage consolidates a collection of IMF-provided CSV files into a single, clean relational database. This is implemented in `creating_database.py`. The script loads all CSV files into a unified SQLite database `imf_data.db`, with each file becoming its own table.

Before insertion, the script applies a lightweight normalization step: column names are lowercased, whitespace and punctuation are replaced with underscores, and column names beginning with a digit are prefixed. These transformations produce a regularized schema while preserving the original IMF structure.

Each cleaned table is inserted using SQLAlchemy's `.to_sql()` function with the `replace` option, ensuring deterministic reconstruction of the database across runs.

4.2 Query-Processing Pipeline

The second stage translates natural-language questions into SQL and generates a narrative answer. This pipeline is implemented in `imf_nlqa.py` and orchestrated via `run_pipeline.py`. It consists of three main steps.

4.2.1 Schema Extraction

At runtime, the system extracts the database schema by enumerating table names and their columns. This information is formatted into a structured text description used within the LLM prompt. Schema serialization prevents the model from hallucinating nonexistent tables or fields.

4.2.2 SQL Generation

The system uses an LLM to generate SQL from the question and schema description. The model is prompted to use only tables and columns listed in the schema, return a single SQL statement, and avoid destructive operations. The generation relies entirely on prompting rather than intermediate representations or retrieval modules.

4.2.3 Execution and Answer Verbalization

Once a query is generated, it is executed against the SQLite database. The resulting dataframe is then supplied to the LLM, which produces a concise, domain-appropriate explanation. The verbalization step is constrained to describe only the retrieved results, ensuring factual grounding.

5 Query Template Generation

To successfully benchmark different models' ability to map natural-language questions to SQL queries, a collection of templatized prompts of varying complexities was created. These templates were later used to generate concrete questions that directly referenced schema elements in the database rather than abstract schema categories. The model's accuracy in synthesizing an SQL solution from these questions forms the foundation of the benchmark and allows for examination of its capabilities.

Templatization was chosen for several reasons. First, it ensured uniqueness between queries. Non-overlapping questions were a priority in the generation process, since uniqueness guarantees that each prompt tests a distinct interaction between the LLM and the database. Templatization also made comparisons easier and encouraged a clearer under-

standing of the relationships being queried, rather than the specific countries or indicators involved.

Second, templatization allowed for straightforward complexity scaling and the efficient creation of more complicated prompts. In text-to-SQL problems, a natural-language question can be broken down according to the SQL elements and operations it requires, which is reflected by the number of `[Insert Operation/Schema]` markers in the template. Additionally, complex queries can often be constructed by combining multiple simple queries. These combinations introduce additional length and depth for the LLM to decipher. As a result, while most extremely hard templates were written by hand, many medium-complexity templates were efficiently generated by combining simpler cases or through additional manual design.

Finally, templatization supports easy replication of the question-generation process, whether for methodological repetition or for expanding the question set. Readers can more easily follow how a template was created and substitute schema elements as needed, compared to interpreting a single highly specific handwritten question.

Each template was written by hand with an emphasis on medium to hard queries. After approximately 80 templates were created, their overall complexity distribution was evaluated before adding more simple and extremely hard queries to balance the set. The goal for each template was to examine a different aspect or relationship within the data. Although authors chose the subject and flow independently, all templates followed the same convention of using `[Insert]` markers to denote replaceable schema elements or SQL operations.

Once the total reached over 100 templates, a full complexity evaluation was conducted, and each template was transformed into a specific natural-language question. Authors selected the schema elements and operations to replace the markers. During this process, the main consideration was ensuring that the template was applicable to the corresponding table. Given the wide variety of country-level data across IMF tables, it was necessary to balance schema diversity with verifying that the targeted data existed for the selected country and in the correct format.

Complexity categorization played a central role in understanding where LLMs struggle and how performance differs across question types. Each query was assigned one of four complexity levels, which served as a comparison metric during evaluation. Complexity was determined using two overarching factors: natural-language difficulty and SQL complexity.

Natural-language difficulty refers to how challenging the question is to interpret, including ambiguity, length, and clarity of schema references. SQL complexity reflects the structural difficulty of the expected SQL, such as the presence of common table expressions (CTEs) or nested queries. For example, calculating an intermediate value such as an average before comparing it to another metric introduces a nested query and therefore increases complexity. Similarly, questions requiring access to multiple unrelated tables receive higher complexity classifications.

Using these considerations, the following guidelines were used to characterize template complexity:

- **Low**: No nested queries, no complex CTEs, and straightforward natural language.
- **Medium**: Contains either a nested query, complex CTEs, or complex natural language.
- **High**: Contains multiple nested queries, complex CTEs, or complex natural language.
- **Very High**: All forms of complexity are present.

6 Code Implementation

The full system is implemented through three main scripts: `creating_database.py`, `imf_nlqa.py`, and `run_pipeline.py`. Together, they form an end-to-end pipeline that (1) builds a clean SQLite database, (2) extracts schema information, (3) generates SQL using a local or remote LLM, (4) executes the query, and (5) produces a natural-language explanation of the results.

Below, each each component is described in detail.

6.1 Natural-Language Query Pipeline

The main logic of the question-answering system is implemented in `imf_nlqa.py`. This script handles schema extraction, SQL generation, query execution, and verbalization.

6.1.1 Schema Extraction

When the script loads, it connects to the SQLite database and constructs a complete schema de-

scription using PRAGMA calls. For each table, it records:

- the table name,
- each column name and its SQLite type.

This schema text is later embedded directly into the LLM prompt so that the model generates SQL grounded in the actual database rather than hallucinating fields.

6.1.2 SQL Generation

All SQL generation is performed through an LLM call, using either:

- a local Ollama model (as the default), or
- a cloud model if available.

The SQL generator strictly enforces:

- no explanations or commentary,
- output must be a single SQL query,
- only tables and columns present in the schema may be referenced.

The prompt positions the model as an "IMF analyst" whose job is to interpret the question and produce SQL only. This constrained prompting strategy substitutes for more complex semantic parsing or validation frameworks.

6.1.3 Query Execution

After SQL is generated, the script executes it directly against the same SQLite connection using `pandas.read_sql_query`. Errors such as missing columns or invalid syntax are caught and returned as strings.

6.1.4 Answer Explanation

The final step is to convert the raw dataframe into a natural-language answer. A second LLM call summarizes the retrieved data, with instructions to:

- avoid referencing SQL syntax,
- avoid making up values not present in the dataframe,
- provide a clear, domain-appropriate explanation.

Separating SQL generation and explanation reduces hallucination risk, since the explanation model always receives the actual numerical output.

6.2 End-to-End Pipeline Script

The script `run_pipeline.py` serves as a lightweight wrapper around the functions defined in `imf_nlqa.py`. It provides a simple command-line interface:

1. load the schema,
2. accept a natural-language question,
3. call the SQL generator,
4. execute the produced query,
5. display both the SQL and the natural-language answer.

This script is primarily intended for testing and demonstration. It reuses the same schema extraction and LLM prompts, ensuring that results are consistent with the core pipeline.

7 Conclusion & Limitations

This project developed an end-to-end pipeline that converts natural-language questions into SQL queries, executes them on a unified IMF database, and produces natural-language answers. It also introduced a set of 110 templated and categorized questions designed to evaluate model behavior across a range of complexity levels. The system showed strong performance on simple lookup queries and demonstrated that schema grounding, canonicalization, and dynamic schema extraction can substantially improve SQL correctness. Results further indicated that LLMs are able to handle a combination of straightforward, multi-table, multi-indicator, and temporally-oriented reasoning queries.

Despite these strengths, the system has several limitations. The evaluation set is still relatively small and may not reflect the full variety of real-world queries. Using SQLite also limits the range of advanced SQL operations that can be tested. In addition, inconsistencies across IMF tables and indicator definitions make reasoning harder for the model, especially when schema structures vary widely between datasets. These issues highlight opportunities for future work, such as expanding the benchmark, exploring more expressive database backends, and improving schema alignment across IMF sources.

References

IMF data (2025) IMF.
Available at: https://www.imf.org/en/data

Lan, W. et al. (2023) Unite: A unified benchmark for text-to-SQL evaluation.
Available at: https://arxiv.org/abs/2305.16265

Lee, Jihyung et al. (2025) DCG-SQL: Enhancing in-context learning for text-to-SQL with deep contextual schema link graph.
Available at: https://arxiv.org/abs/2505.19956

Rajkumar, N., Li, R. and Bahdanau, D. (2022) Evaluating the text-to-SQL capabilities of large language models.
Available at: https://arxiv.org/abs/2204.00498

Yu, T. et al. (2019) Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task.
Available at: https://arxiv.org/abs/1809.08887

Zhang, B. et al. (2024) Benchmarking the text-to-SQL capability of large language models: A comprehensive evaluation.
Available at: https://arxiv.org/abs/2403.02951

Zhang, K. et al. (2023) ReFSQL: A retrieval-augmentation framework for text-to-SQL generation.
Available at: https://aclanthology.org/2023.findings-emnlp.48/